

Spotify Music Graph — Project Plan

Context

Inspired by a YouTube video that maps all of Wikipedia as a graph (igraph + DrL layout + Leiden community detection, with careful "valid article" / "valid link" scoping to keep the graph meaningful rather than noisy), the user wants to build the same style of artifact from their own Spotify library: songs as nodes, edges based on shared artist/genre/album/playlist co-occurrence.

This is a **personal learning project** — the user will do all implementation themselves. The user has **some prior coding experience** (scripts/a course, but nothing at this scale) and wants **maximum depth**: explicit languages, explicit libraries with install commands, concepts explained in plain terms, and free resources linked at each new tool/concept. **Budget is \$0** — every recommendation below is chosen to avoid paid services; optional paid upgrades are called out separately and are never required.

Confirmed decisions from the user: - **Scale**: Phase 0 found 7,990 unique tracks in the current library (2026-08-03). Design target going forward is **at least 20,000 nodes**, set deliberately above the current actual count as a robustness target for library growth (more listening, more playlists) — evaluate Phase 6 (layout) and Phase 8 (interactive explorer performance) against 20k, not against today's 7,990. - **Node scope**: broadest option — Liked Songs + owned/followed playlists + top tracks (all time ranges) + recently played. - **Edges**: inclusive/dense to start (any shared artist, genre, album, playlist counts), explicitly expecting to need to tighten this later. - **Deliverable**: static rendered image first, interactive explorer after.

Key external constraint: Spotify deprecated several Web API endpoints in Nov 2024 for apps without pre-existing extended quota — notably **Audio Features, Audio Analysis, Recommendations, and Related Artists**. Confirm actual access early (a single test call) rather than assuming. Genre is **artist-level only** (`/artists` → `genres[]`), never per-track — this must be joined in, not read off the track object.

Cost summary: this plan is designed to cost \$0

Need	Free tool	Notes
Spotify data access	Spotify Web API (Development Mode)	Free for personal apps; capped at 25 registered users, unlimited for your own use within rate limits
Language runtime	Python 3.x	Free, python.org
Spotify API wrapper	<code>spotipy</code> (pip package)	Free, MIT license
Graph engine + layout + community detection	<code>python-igraph</code> (pip package)	Free, GPL; Leiden algorithm is built in, no extra package needed
Static render + manual layout tuning	Gephi (desktop app)	Free, open source, no coding required for this step
Interactive explorer	<code>pyvis</code> (pip package)	Free; generates a self-contained interactive HTML file, no JavaScript required
Editor	VS Code	Free
Version control / free hosting for the final HTML	Git + GitHub + GitHub Pages	Free
Compute	Your own laptop	A personal library (likely hundreds to low tens of thousands of tracks) is small enough to run entirely locally — no cloud costs

Optional paid extras (not required, only mention if you want them later): - Custom domain name instead of a free `github.io` URL: ~\$10–15/year. - Hosted graph database (Neo4j Aura) if you ever outgrow an in-memory igraph graph: has a free tier; paid tiers start around \$65/month — extremely unlikely to be needed at personal-library scale, skip unless you hit a real wall. - A cloud VM if your laptop struggles: a few dollars/month — again, unlikely necessary given the scale involved.

Tooling overview (the "why" behind each choice)

Two languages, used for different jobs, with a way to avoid the second one entirely if you want: 1. **Python** — all data collection, cleaning, graph construction, and community detection. This is the core of the project (Phases 0–7). 2. **JavaScript/HTML** — only if you choose the "build your own" path for the interactive explorer in Phase 8. The **recommended default** is `pyvis`, which writes the JS/HTML for you

from Python, so you can complete the entire project without hand-writing any JavaScript. Treat hand-rolling the JS version as an optional stretch goal, not a requirement.

Environment setup (do this once, at the start): - Install Python 3 (macOS: `brew install python` via [Homebrew](#), or the installer from python.org). - Create an isolated environment for this project so its packages don't collide with anything else on your machine: `python3 -m venv venv` then `source venv/bin/activate`. Docs: [venv — Python docs](#). - Install packages inside that environment with `pip install <name>`. - Recommended editor: [VS Code](#) with the Python extension — free, gives you inline errors and autocomplete.

Phase 0 — Find out the actual scale

Goal: get a real number for how many unique tracks you're dealing with, before deciding anything about layout algorithms or rendering strategy — this one number changes what's realistic later.

What to build: a small script that logs into the Spotify API and counts tracks across your four sources, without storing anything yet.

Libraries: `spotipy` (`pip install spotipy`) — the standard, free Python wrapper around the Spotify Web API. Docs: [spotipy.readthedocs.io](#).

Concept — OAuth: Spotify won't let a script read your library with just a password; it uses OAuth, where you register an "app" (free, takes 5 minutes at [developer.spotify.com](#)), get a Client ID/Secret, and `spotipy` handles the browser-login handshake for you (`spotipy.oauth2.SpotifyOAuth`) — you just provide the scopes (permissions) you're requesting. Store the Client ID/Secret in a `.env` file (never commit this to git) and load it with `python-dotenv` (`pip install python-dotenv`) rather than hardcoding it in your script.

What this number tells you: - Under ~5,000 tracks: any layout algorithm renders in seconds. Skip DrL — it exists for Wikipedia's ~6M-article scale, not a personal library. - 5,000–20,000+: still tiny next to Wikipedia's scale, but worth timing a layout run before committing to a specific tool.

Update, 2026-08-03: Phase 0 found 7,990 tracks — in this middle band. The user has since set a **20,000-node design target** (deliberately above the current count, for headroom as the library grows), so Phase 6 should budget real time for timing a layout run at that scale rather than assuming the "under 5k" trivial case, even though today's actual number is comfortably below 20k.

Verification: script prints one number (deduped track count) plus a breakdown by source (liked / playlists / top tracks / recently played).

Phase 1 — Raw data collection

Goal: pull everything you need once, cache it locally, and never hit the API again unless you actually need fresh data.

Libraries: `spotipy` (same as Phase 0), plus either the built-in `json` module (simplest — write each API response to a `.json` file) or `sqlite3` (built into Python, no install needed — better if you want to query the raw data later rather than re-parsing JSON every time).

Scopes needed (tell `SpotifyOAuth` to request these): `user-library-read`, `playlist-read-private`, `user-top-read`, `user-read-recently-played`.

Sources to pull: - `/me/tracks` — Liked Songs. - `/me/playlists`, then playlist items for each — your own and followed playlists. - `/me/top/tracks` — call three times, once per `time_range` value (`short_term`, `medium_term`, `long_term`). - `/me/player/recently-played` — capped at ~50 by the API, so expect heavy overlap with the sources above. - `/artists?ids=...` — batch-fetch (up to 50 IDs per call) for every unique artist ID you see, since **genre lives on the artist, not the track**.

Concept — pagination & rate limits: these endpoints return 50 items per page with a `next` URL for the following page; `spotipy` exposes this via `sp.next(results)`. Loop until `next` is `None`. Spotify also rate-limits you (HTTP 429) if you call too fast — `spotipy` raises an exception with a `Retry-After` header when this happens; catch it and sleep for that many seconds, or just add a small delay between calls during your first pass.

Verification: raw cache contains the same total track count Phase 0 found, plus a genre list for every artist ID referenced.

Phase 2 — Node table

Goal: turn the raw, messy JSON dump into one clean row per unique track — this is a standard data-cleaning step, no new APIs involved.

Libraries: `pandas` (`pip install pandas`) is the natural fit here — lets you load your cached JSON into a table and dedupe/join with a few lines, rather than writing manual loops. Not required (you could do it with plain dicts/lists), but recommended since it makes the next two phases much easier. Docs: pandas.pydata.org.

Columns to build per track: - `id`, `name`, `artist id(s)` (ordered — first is primary), `album`, `release date`, `duration`, `popularity`. - Joined-in genres: union of all its artists' genres (decide whether to weight primary-artist genres higher than featured-artist genres — revisit in Phase 3 if needed). - Provenance flags: which source(s) it came from. Cheap to keep now, potentially useful later as a filter or edge signal.

Concept — dedup key: the same recording can appear under multiple Spotify track IDs (single vs. album version, remaster, deluxe reissue). If that matters to you, ISRC (an international recording code, present on the track object) is a better dedup key than track ID — decide whether to collapse these into one node or keep them distinct.

Verification: row count matches Phase 0's number (or you can explain every discrepancy, e.g. ISRC collapsing removing some duplicates).

Phase 3 — Edge signals (compute separately from edge inclusion)

Goal: for every meaningful pair of tracks, compute *why* they might be connected — as several separate scores, not one blended number. This is the phase most likely to need revisiting given the "inclusive now, tighten later" edge decision, so this separation is what makes tightening later a **parameter change, not a rewrite**.

No new libraries — this is plain Python/pandas logic over the node table from Phase 2 (e.g. grouping tracks by artist ID, by genre, by album, by playlist, and recording which tracks land in the same group).

Signals to compute per pair: - shared primary artist (binary, or a count if multiple co-artists) - shared featured/secondary artist - shared genre count — consider inverse-frequency weighting: two tracks sharing "pop" means much less than two sharing a niche microgenre, the same way a Wikipedia hub page linking somewhere is less meaningful than a niche page doing so - shared album - shared playlist co-occurrence (count of playlists both appear in)

Store each of these as a **separate column on the edge**, not collapsed into one weight yet — that collapsing happens in Phase 4, where it's cheap to change.

Known problem to expect (flagging proactively since dense/inclusive was the explicit choice): naive shared-genre edges on a popular genre will create a near-complete subgraph among everything tagged that genre — the same failure mode as a Wikipedia hub page. Two standard fixes, apply when you actually hit this: (a) inverse-genre-frequency weighting (rarer shared genre = stronger edge), (b) cap edges per node to the top-k highest-weighted candidates instead of including every match.

Verification: spot-check a handful of track pairs by hand — do the computed signals match what you'd expect for songs you know well?

Phase 4 — Edge inclusion policy

Goal: turn signals into an actual edge list.

No new libraries. Combine the Phase 3 signal columns into one weight (a simple weighted sum is a fine start — e.g. `weight = 3*shared_artist + 1*shared_genre + 2*shared_album + 2*shared_playlist`) and apply either a minimum-weight threshold or a per-node top-k cap (keep only each node's N strongest connections). **Keep the weights and threshold as variables at the top of your script**, not buried in logic — this is exactly where "inclusive & dense" vs. "tight & curated" becomes a dial you can turn without touching the signal-computation code from Phase 3.

Verification: check total edge count and degree distribution (a histogram of edges-per-node — `pandas / matplotlib` can do this in a couple of lines). A healthy graph has a spread of degrees; a hairball looks like most nodes sitting at very high degree.

Phase 5 — Graph construction + community detection

Goal: turn your node table and edge list into an actual graph object, and detect clusters within it.

Library: `python-igraph` (`pip install python-igraph`) — note the pip package name is `python-igraph` but you `import igraph` in code. Docs: python.igraph.org.

Concept — building the graph: `igraph.Graph()` takes a list of edges (pairs of node indices) and lets you attach attributes to nodes (name, genre, artist) and edges (weight). This is the same tool the Wikipedia video used.

Concept — Leiden community detection: this groups nodes into clusters that are more densely connected internally than to the rest of the graph — here, "sonic neighborhoods" of your library, rather than Wikipedia topic clusters. It's built directly into `igraph` as a method — `graph.community_leiden(objective_function=..., weights=..., resolution=...)` — so no separate package is needed (older tutorials mention a separate `leidenalg` package; with a reasonably recent `python-igraph` you don't need it, though `leidenalg` remains available if you want finer control over its parameters later).

Verification: print community sizes — a few large, meaningful-looking clusters (not one giant blob, not thousands of singleton clusters) is the signal this phase worked. The `resolution` parameter controls how fine- or coarse-grained the clusters are if your first attempt looks off.

Phase 6 — Layout (recommended: hand this off to Gephi, not code)

Goal: compute (x, y) positions for every node so that connected/ similar nodes end up near each other visually.

Recommended path: rather than fighting layout-algorithm APIs and plotting libraries in Python, **export your graph from igraph to a GraphML or GEXF file** (`graph.write_graphml("graph.graphml")` — igraph supports this natively) and open it in **Gephi** (free desktop app, gephi.org). Gephi has ForceAtlas2, Fruchterman- Reingold, and other layouts available as GUI buttons, with sliders for their parameters and a live preview — this avoids a fair amount of fiddly plotting-library setup for a "some experience" skill level, and gives you instant visual feedback while tuning.

Why not DrL, and when to reconsider: DrL exists specifically for graphs at Wikipedia's ~6M-node scale. A personal library doesn't need it at typical sizes — Gephi's ForceAtlas2 will converge quickly and look good well past the 20k-node design target set after Phase 0 (2026-08-03). Still, since 20k is now the explicit bar rather than an aspirational edge case, **actually time a ForceAtlas2 run at (or synthetically padded to) ~20k nodes before calling this phase done**, rather than assuming it's fine by extrapolation. (DrL remains available in igraph directly, as `graph.layout_drl()`, if that timing test comes back slow — treat it as the fallback, not the default.)

Verification: layout completes in a reasonable time (seconds to low minutes in Gephi) and visibly doesn't look degenerate (everything piled at one point, or a perfect uniform grid).

Phase 7 — Static render

Goal: produce your first real "did this work" checkpoint — a static image — before investing in an interactive app.

Recommended path: still inside Gephi — color nodes by the Leiden community attribute you brought in from Phase 5 (Gephi's "Partition" coloring panel does this directly from an attribute column), then use Gephi's built-in "Preview" tab to export a high-resolution PNG/SVG/PDF. No additional coding required for this step.

Alternative (more coding, more control): if you'd rather stay in Python end-to-end, `igraph` can plot directly (`igraph.plot(graph, layout=..., vertex_color=...)`), though it requires the `pycairo` package, which can be fiddly to install on macOS — Gephi avoids this entirely, which is why it's the recommended default here.

Verification: the image itself — do visible clusters correspond to things you'd recognize (a genre, an artist phase, a specific playlist)? This is also your checkpoint to decide whether Phase 3/4's edge weights need tightening before moving on.

Phase 8 — Interactive explorer

Goal: a browser-based version you can pan, zoom, click nodes, and search — without needing to learn a JavaScript framework first.

Recommended library: `pyvis` (`pip install pyvis`) — a Python library that wraps the JS library `vis.js` and **writes a complete, self-contained interactive .html file for you from a Python graph object**. You give it nodes/edges/colors/positions from your `igraph` graph (`pyvis` works most directly with `networkx` graphs, so converting your `igraph` graph to `networkx` first is a small, well-documented step — or you can feed `pyvis` nodes/edges directly without `networkx` at all) and it handles all the JS/HTML/CSS. Docs: pyvis.readthedocs.io. This means you can finish the entire project without hand-writing JavaScript.

Stretch goal (optional, only if you want to learn web dev along the way): for more visual control or better performance at larger scale, a hand-built page using **sigma.js** (free, MIT license, WebGL-based, built specifically for network visualization — sigmajs.org) is the natural next step up from `pyvis`. Treat this as a "if I want to go further" upgrade, not a requirement to finish the project.

Free hosting for the final result: once you have a single `.html` file (from `pyvis` or otherwise), you can publish it for free via GitHub Pages (docs.github.com/en/pages) — push it to a GitHub repo (a free account), enable Pages in the repo settings, and you get a shareable URL with no server or hosting cost.

Verification: can click a node and see its neighbors highlight; can search/filter by artist or genre; interaction stays smooth while panning/ zooming.

Notes / things to keep in mind, not phases

- Prerequisite the user handles themselves: register a Spotify Developer app (free) and get OAuth working — confirm at that point whether Audio Features/Related Artists are actually available to the app, since that changes what signals Phase 3 has available.
- Since edges started inclusive/dense by explicit choice, expect to return to Phase 3/4 at least once after seeing Phase 7's static render — the plan is deliberately structured (signals separate from inclusion policy) so that's a config change, not a redo.
- Total new tools to learn, in rough order of appearance: `spotipy` → `python-dotenv` → `pandas` → `python-igraph` → Gephi (GUI, no code) → `pyvis`. Each is free, each has docs linked above, and each is introduced only in the phase where you actually need it.